Security Controls in Shared Source Code Repositories

Julio Pochet Edmead

CSD-380 • DevSecOps

Module 11.2 Assignment

October 5, 2025

Introduction

Shared repositories like GitHub and GitLab make teamwork faster and more efficient. But when multiple developers share the same space, protecting that environment becomes just as important as collaboration itself. This presentation highlights practical ways to keep shared repositories secure — without slowing down productivity.

Why Security Controls Matter

Sensitive Data Protection

Source code often contains sensitive data like keys, tokens, or internal logic.

Attack Prevention

Attackers target repos to steal credentials or insert malicious code.

Reputation & Compliance

Security controls prevent data leaks and protect company reputation.

Standards Compliance

They also keep projects compliant with security and privacy standards.

Access Control & Permissions

- Give team members access only to what they need.
- Use **role-based permissions** and require **multi-factor authentication (MFA)**.
- Review access lists regularly and remove unused accounts.
- Never share credentials or embed them directly in code.

Branch Protection & Code Reviews

Branch Protection

Protect main branches to stop unauthorized or accidental commits.

Pull Requests

Require pull requests and at least one peer review before merging.

Commit Signing

Enable **commit signing** to verify the source of changes.

Automated Scans

Automate scans on pull requests to detect vulnerabilities early.

Secrets Management

- Avoid storing credentials or API keys in plain text.
- Use tools like **GitHub Secrets**, **Vault**, or **AWS Secrets Manager**.
- Rotate keys regularly and store them outside of your codebase.
- This reduces the risk of leaks if a repository is ever compromised.

Automated Scanning & Monitoring

Integration

Integrate **static analysis** and **dependency scanning** into your CI/CD pipeline.

Tools

Tools like **Snyk**, **Checkmarx**, and **SonarQube** catch vulnerabilities automatically.

Alerts

Set up notifications for risky commits or permission changes.

Consistency

Continuous scanning keeps security consistent without manual effort.

Developer Awareness & Culture

Security tools work best when everyone on the team understands their role.

Offer quick security refreshers to help developers spot potential issues.

Encourage open communication when someone notices a risk.

A strong security culture protects both code and collaboration.

Conclusion

> Security controls keep repositories safe without slowing innovation. With proper access, secret management, and continuous scanning, teams can build securely while working efficiently. The goal is simple: protect the code, the team, and the people who rely on your software.

References

Harness. (2025, June 26). *Best practices for securing code repositories.* Harness DevOps Academy. Retrieved from https://www.harness.io/harness-devops-academy/secure-code-repositories-best-practices

---

Rose, J. (2025, June 11). *12 best practices for secure code repositories (2025 guide).* Checkmarx Blog. Retrieved from https://checkmarx.com/supply-chain-security/repository-health-monitoring-part-2-essential-practices-for-secure-repositories/